

COMS 4444: Birthday Cake Group 3

Andrew Chung, Christophe Charles, Axel Browne, Akshat Bhandari

October 20 2025

Contents

1	Problem	3
2	Cake Designs	3
2.1	Easy Cake - Electric Guitar	3
2.2	Challenging Cake - Zia	3
2.3	Fun Cake - Diamond Sword	4
2.3.1	Judging	4
3	Early Design - Pair of Points Approach	4
3.1	Limitations	5
3.1.1	Computational Efficiency	5
3.1.2	Cut Accuracy	5
4	Intermediate Design - Angular Binary Search	5
4.1	Algorithm Overview	6
4.2	Pseudocode	7
4.3	Remaining Limitations	8
5	Final Design - Recursion and Buckets	9
5.1	Recursion	9
5.2	Recursion Pseudocode	11
5.3	Parallelization	12
6	Tournament Results	12
6.1	Overall Tournament Rankings	14
6.2	6.1 Benchmark Cakes Analysis	16
7	Conclusion	19
8	Future Direction	19
9	Summary of Contributions	19
10	Acknowledgment	20

1 Problem

We were tasked with developing a solution to the fair cake-cutting problem. Given a cake:

1. Divide the cake into equal areas for N children, such that each piece comprises $\frac{1}{N}$ of the total area, within a narrow error.
2. Generate cuts that produce evenly split crust-to-interior ratios across all pieces. Where crust is defined as the outer edge and the interior is the inner part of the cake.

We designed three cakes—a simple benchmark, a party-appropriate design, and a complex geometric one—each constrained to have at least 50% interior material, interior angles of at least 15° , and piece areas between 10 and 50 cm^2 . Designs were also evaluated by vote for their appeal as real birthday cakes.

2 Cake Designs

We used Desmos to visualize and design cake geometries before implementation. This tool allowed us to position points precisely and ensure they could be traced sequentially in the CSV format. If a designed cake exceeded or fell short of size constraints, we could uniformly scale all points by a constant factor to meet requirements.

We designed three cakes according to the problem specifications: an easy cake (Electric Guitar), a challenging cake (Zia Symbol), and a fun cake (Minecraft Diamond Sword).

2.1 Easy Cake - Electric Guitar

The electric guitar was designed by combining multiple polygons to create a simple yet distinctive shape. However, testing revealed significant challenges. The cake contained narrow sections, particularly around the neck area, where crust surrounded the interior on both sides. This geometric constraint made it difficult to produce pieces with balanced crust-to-interior ratios, especially when cutting for larger numbers of children. Consequently, the guitar proved to be a more challenging cake than initially intended. Anything past 5 children proved difficult to handle in terms of crust to pie ratio with our algorithm.

2.2 Challenging Cake - Zia

The inspiration to use the "Zia Symbol" came from three sources of inspiration. The cake has several properties. It is symmetrical along three axes. It has a "round" interior with many vertices. The Zia has a "jagged" crust along its four sides.

The best solution to divide this cake is likely one that takes advantage of its radial/rotational symmetry. But, it is a non-trivial task for a cake-cutting

agent to divide the "jagged" crust fairly, and may have to make difficult trade-offs between maintaining cut size homogeneity and piece-cut-ratio consistency.

2.3 Fun Cake - Diamond Sword

The diamond sword was designed directly from the iconic Minecraft Game, and is known as one of the higher-tiered weapons in the game. The original design was scaled to have an area of 312 cm² so that for the default children, it would have medium-sized pieces. The shape itself is vertically symmetric, but the orientation in the design make it symmetric across a diagonal line. Since the design was "pixelized" with precise edges, we initially thought it would be easier to divide. However, the sword handle was just small enough for the crust from both sides to touch, which made it more difficult to maintain the desired ratio when cutting.

2.3.1 Judging

We decided to submit this design for judging as a set cake. Those who recognized the Minecraft theme gave us high ratings (9/10), but overall our cuts received a lower average score (4.67/10). We chose to make 11 slices because that number provided the greatest increase in portions above the default while keeping the crust ratio deviation low (4.59) and the computation time within range. In hindsight, we should have selected a design that made the parents' task easier, specifically by choosing an even number of slices so the first cut could divide the sword evenly in two.

We also want to mention that the judging by the artist seemed to break the spirit of the competition, as the idea was to make a cake that a child would enjoy. It was not based on whether it is "unusual" enough. Unless they believed that the cake was not unusual enough for a child, we would take this criticism with a grain of salt.

3 Early Design - Pair of Points Approach

During initial design discussions, we explored various approaches including divide-and-conquer strategies, angular optimization, and area-based cutting methods. We ultimately implemented a straightforward solution that searched through pairs of points along the cake perimeter.

To determine valid cuts, we introduced varying thresholds for both the piece area and crust-to-cake ratio. This approach resulted in a time complexity of $O(Xjkn^2)$, where j is the number of area thresholds, k is the number of ratio thresholds, X is the number of children, and n is the number of perimeter points. Additionally, the algorithm found one piece per iteration, targeting pieces of size $\frac{1}{X}$ at each step. This design was both inefficient and inaccurate.

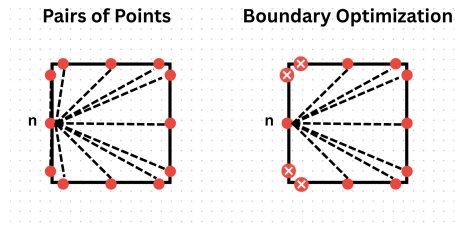


Figure 1: For each point n , we check all other points along the perimeter. To optimize slightly, we filter out points that form angles too narrow from n .

3.1 Limitations

3.1.1 Computational Efficiency

The fourth-order time complexity severely limited scalability. Adding just 10 additional perimeter points caused exponential increases in runtime, since we evaluated every point pair across all area and ratio thresholds. We were restricted to testing only 50 perimeter points without exceeding the one-minute time limit.

3.1.2 Cut Accuracy

The geometric constraint of evenly-spaced perimeter points created fundamental accuracy issues. We were not guaranteed to find points that accurately divided arbitrary cake geometries, even as area and ratio tolerances increased. The discretization of the perimeter restricted our ability to find optimal cut positions.

4 Intermediate Design - Angular Binary Search

Ultimately, we were left with two major issues: time complexity and general flexibility. The second iteration of our approach successfully addressed these two challenges. We implemented an angular binary search where, rather than checking all pairs of points across the perimeter, we would select a point on the perimeter and perform an angular binary search at that point to maximize the cut area. We still generated a finite set of perimeter points, but instead of checking all pairs, we used binary search to identify the second point. Additionally, since we could determine area during the binary search and filter the best cuts based on ratio after finding “perfect” area cuts, we no longer needed area and ratio tolerance parameters. This reduced our time complexity from $O(jk(n^2))$ to $O(n \log(p))$, where X is the number of children, p is the number of points in the search space, and n is the number of perimeter points.

4.1 Algorithm Overview

The algorithm performs an angular binary search for each point on the perimeter based on a larger imaginary circle surrounding the cake. The initial line is always drawn from the centroid of the current piece, and the binary search continues until the search space becomes too small. During the binary search, we track the best area cuts, returning only those that meet our target area. The ratio filter is applied afterward, as correct area is a prerequisite for ratio optimization.

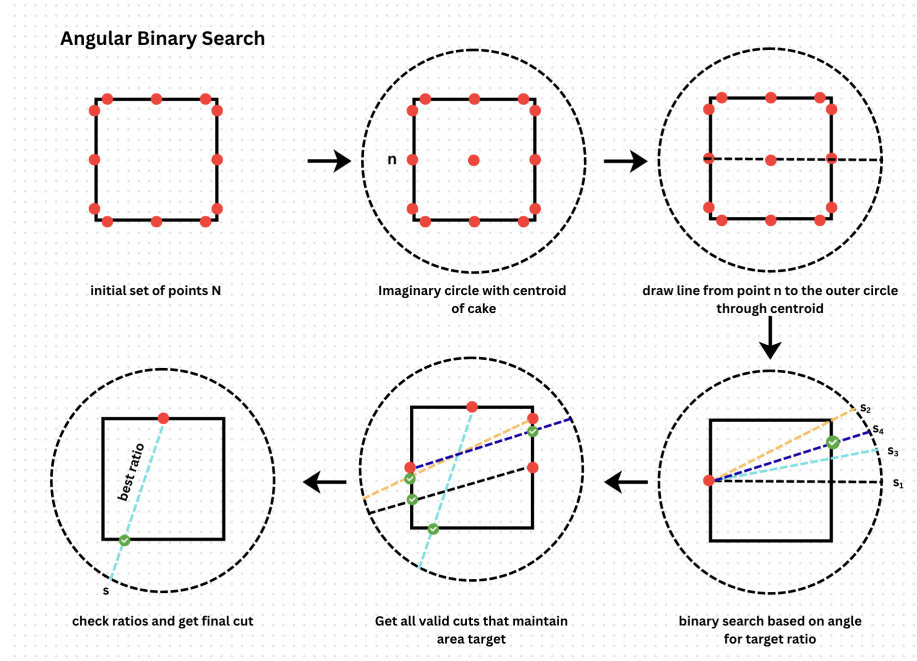


Figure 2: Angular Binary Search. Search is performed using each point on the cake perimeter to an outer circle.

This approach represented a major improvement, increasing the speed of valid cut verification and generating more accurate cuts since the second point was no longer constrained to a finite set of candidates. We increased the number of perimeter points checked from 50 to 500, a 10-fold increase in sampling density.

4.2 Pseudocode

The main binary search is as follows:

Algorithm 1 BinarySearchCut

```
function BINARYSEARCHCUT(cake, point, targetArea, piece)
  angle  $\leftarrow$  angle from point to piece centroid
  left, right  $\leftarrow$  angle  $-2\pi$ , angle  $+2\pi$ 
  while right - left > 0.001 do
    mid  $\leftarrow$  (left + right) / 2
    endpoint  $\leftarrow$  FindPoint(point, mid, piece)
    pieces  $\leftarrow$  cake.cut(point, endpoint)
    areaDiff  $\leftarrow$  min area difference from targetArea
    if areaDiff < bestDiff then
      bestDiff  $\leftarrow$  areaDiff, bestPoint  $\leftarrow$  endpoint
    end if
    if min piece area < targetArea then
      left  $\leftarrow$  mid
    else
      right  $\leftarrow$  mid
    end if
  end while
  if bestDiff < 0.20 then
    return (bestPoint, bestDiff)
  end if
end function
```

The algorithm performs an angular binary search, where the initial cut is devised by drawing a line through the centroid of the current piece. The search space is initialized as a full rotation ($\pm 2\pi$ radians) around this initial angle. We then iteratively narrow the search space by evaluating the area of resulting pieces at the midpoint angle. If the smaller piece is below the target area, we search angles that produce larger cuts; otherwise, we search angles that produce smaller cuts. The search terminates when the angular range becomes sufficiently small (0.001 radians). Throughout the search, we track the best cut found so far, returning it only if the area difference is below our tolerance threshold of 0.20 square units. The area tolerance threshold of 0.2 cm^2 was empirically determined through testing to balance accuracy with the physical error limit of $\pm 0.5 \text{ cm}$. This represents approximately 40% of the allowable error margin, providing a safety buffer while still producing cuts that fall within acceptable tolerances for the target piece areas. We did notice that anything greater than 0.24 would lead to inaccurate areas.

Algorithm 2 FindPoint

```
function FINDPOINT( $p, \theta$ , piece)
  ray  $\leftarrow$  line from  $p$  at angle  $\theta$  extending distance 100
  intersection  $\leftarrow$  ray  $\cap$  piece.boundary
  if single Point then
    return intersection
  else
    return closest point in intersection to  $p$ 
  end if
end function
```

The above is a helper function that finds the intersection point of a ray with the piece boundary, which serves as the second endpoint of a potential cut line.

Algorithm 3 FindValidCuts

```
function FINDVALIDCUTS(cake, perimPoints, targetArea, origRatio, piece)
  cuts  $\leftarrow$  empty list
  for point in perimPoints do
    (endpoint, areaDiff)  $\leftarrow$  BinarySearchCut(cake, point, targetArea,
    piece)
    pieces  $\leftarrow$  cake.cut(point, endpoint)
    ratioDiff  $\leftarrow$  max ratio difference from origRatio
    cuts.append((point, endpoint, ratioDiff))
  end for
  sort cuts by ratioDiff
  return list of (point, endpoint) pairs
end function
```

Once the cuts are found, we calculate the ratios of the resulting two pieces, getting the ratio difference of each piece in comparison to the target ratio. We then create a list of cuts that contain the maximum ratio diff of the two pieces and sort a list based on best to least ratio differences.

4.3 Remaining Limitations

Despite these improvements, the approach contained a significant limitation: it iteratively searched for pieces of size $\frac{1}{X}$. This proved problematic for complex cake geometries, particularly those with large crust-to-cake ratios. The greedy constraint guarantees that the first cut produces a piece with imbalanced crust distribution. More generally, better cuts may exist at each iteration by considering different piece area ratios. Rather than always dividing for a single child, allocating larger pieces to multiple children could lead to superior cuts in subsequent iterations. This greedy approach resulted in inaccurate final ratios.

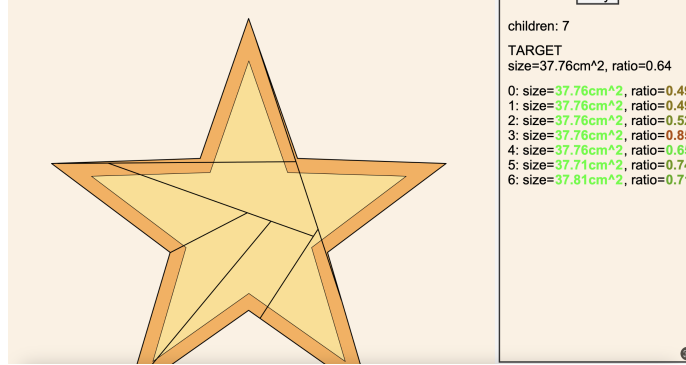


Figure 3: A ratio of 0.85 was achieved for one of the pieces, which was the first “crusty piece” cut leading to high std across the ratios.

We initially attempted to mitigate the “crusty first piece” problem by forcing the first cut to achieve an even area split. However, this failed for very large or complex shapes where even the remaining two pieces could contain excessive crust. A more fundamental solution was needed.

5 Final Design - Recursion and Buckets

Ultimately, we identified one remaining limitation: the inability to evaluate area ratios beyond simple $\frac{1}{X}$ splits at each iteration. We addressed this by combining our angular binary search with a recursive approach that evaluates multiple candidate area ratios at each cut decision. For each ratio candidate, valid cuts are identified and stored, allowing us to select the best cut across the entire ratio space rather than committing to a single $\frac{1}{X}$ split. This approach was further enhanced through parallelization, enabling efficient exploration of the ratio space. This solution substantially mitigates our previous limitation, allowing the algorithm to identify appropriate cuts at each step by adapting to the varying sizes of pieces at cut time.

5.1 Recursion

The recursive division strategy operates by decomposing the cake-cutting problem hierarchically. At each level of recursion, the algorithm identifies the best acceptable cut for the current piece, then delegates the remaining cutting tasks to subsequent levels. The recursion follows a clean divide-and-conquer structure. Given a cake piece and a target number of children to serve, the algorithm first determines the best cut among all candidate area ratios. Once a cut is made, producing two sub-pieces, the algorithm calculates how many children each sub-piece should serve based on their respective areas relative to the target area per child.

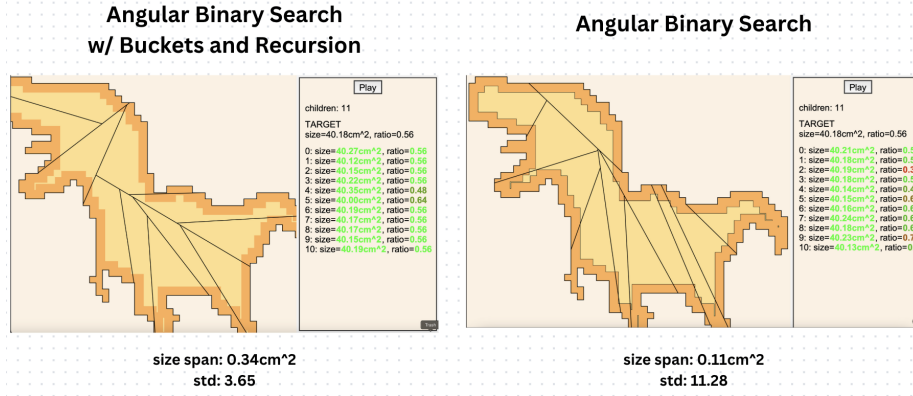


Figure 4: Difference in outcomes when utilizing a bucket to check all ratio cuts, vs just finding a piece for a child each time. It is also interesting to note that recursive cuts tend to be more symmetrical, indicating that even area ratio cuts may be favored.

The algorithm then recursively applies itself to each sub-piece with its allocated number of children. This ensures that the division remains proportional to area throughout the recursion tree, maintaining fairness at every level of decomposition.

The base case occurs when a piece is assigned to a single child (or fewer), at which point no further cuts are needed. Importantly, by evaluating multiple area ratios at each recursive level—rather than committing to a single $\frac{1}{X}$ split, the algorithm can adapt to the geometry of each piece and find great cuts at each step. The parallelization of ratio evaluation across recursive levels further accelerates this search process, allowing the algorithm to explore the solution space more thoroughly without proportional increases in runtime.

5.2 Recursion Pseudocode

Algorithm 4 RecursiveBucketCut

```

function RECURSIVEBUCKETCUT(cake, piece, numChildren, cuts)
  if numChildren  $\leq 1$  then
    return
  end if
  ratios  $\leftarrow [i/\text{numChildren} \mid i \in 1 \text{ to } \lfloor \text{numChildren}/2 \rfloor]$ 
                                      $\triangleright$  Find best cut by evaluating ratios in parallel
  bestCut, bestSplit  $\leftarrow$  BestCutFromRatios(cake, piece, ratios)
  if bestCut is None then
    return
  end if
  cuts.append(bestCut)
  cake.cut(bestCut)
                                      $\triangleright$  Recursively split both pieces
  left  $\leftarrow$  round(bestSplit[0].area / targetArea)
  right  $\leftarrow$  numChildren - left
  if left  $> 1$  then
    RecursiveBucketCut(cake, bestSplit[0], left, cuts)
  end if
  if right  $> 1$  then
    RecursiveBucketCut(cake, bestSplit[1], right, cuts)
  end if
end function

```

The recursive bucket cutting strategy divides the cake hierarchically by repeatedly splitting the largest piece. At each level, the algorithm evaluates multiple potential cut ratios in parallel to find the cut that best preserves the original piece's aspect ratio. The number of child pieces assigned to each split is determined by their relative areas compared to the target area per child.

The algorithm begins by computing potential ratios as fractions of the total number of children to allocate. For example, if cutting for 4 children, potential ratios are $[0.5, 0.75]$, representing cuts that produce pieces for 2 or 3 children respectively. Each ratio is evaluated in parallel by finding the best cut via binary search, and the cut with the smallest ratio deviation is selected.

Once a cut is made, the resulting two pieces are assigned child counts based on their areas. If a piece requires more than one child allocation, it is recursively subdivided. This continues until all pieces are sized for individual children. The parallel evaluation of ratios accelerates the search, and the recursive structure naturally balances the cake division while maintaining aspect ratio consistency across all pieces.

5.3 Parallelization

The addition of multiple area ratio evaluations at each recursive level introduced significant computational overhead. To mitigate this, we implemented parallelization at two levels: (1) ratio evaluation parallelization and (2) recursive task parallelization.

At each recursive level, the algorithm evaluates multiple candidate ratios to determine the best cut. Rather than sequentially testing each ratio, we distribute these evaluations across worker processes using a process pool executor. Each worker independently performs a binary search for the best cut at its assigned ratio, computes the area of the resulting pieces, and calculates quality metrics. Once all workers complete, the main thread collects results and selects the cut with the best ratio preservation. This approach scales well since ratio evaluations are independent with no shared state beyond the input piece geometry.

After a cut is made, the algorithm must recursively process two resulting sub-pieces. Since these sub-problems are independent, we parallelize them using a second process pool executor with two worker slots. Each worker recursively divides its assigned piece and returns a list of cuts. The main thread waits for both workers to complete before merging their results. This recursion-level parallelization becomes increasingly effective as the branching factor grows, leading to potential $2\times$ speedups at each level.

Parallelization introduces overhead from process creation, inter-process communication, and deep copying of cake state. These costs are justified when computational work exceeds communication overhead. For small cakes or few children, this overhead may dominate; however, for larger cakes with many children requiring deep recursive trees, parallelization provides substantial speedup.

Parallel vs Sequential Timing Comparison

Designs	Parallel (s)	Sequential (s)
Group9-Dino.csv (11 pieces)	176.589	428.499
rectangle.csv (10 pieces)	83.755	135.894
minecraft_sword.csv (10 pieces)	113.533	159.944
figure_eight.csv (10 pieces)	78.039	176.010
horseshoe.csv (12 pieces)	111.675	288.550

6 Tournament Results

Ten algorithms were evaluated across benchmark and player-submitted cakes, measuring **runtime**, **ratio standard deviation**, and **cut validity**. Each algorithm was tested under identical geometric and child-count conditions to assess fairness, efficiency, and robustness.

Group 3's recursive bucket algorithm demonstrated the most balanced overall performance, combining speed, low variance, and geometric adaptability. It

consistently appeared on the Pareto frontier between fairness and runtime, outperforming both purely heuristic and precision-heavy approaches. This analysis was done after all the cakes that were invalid were taken out.

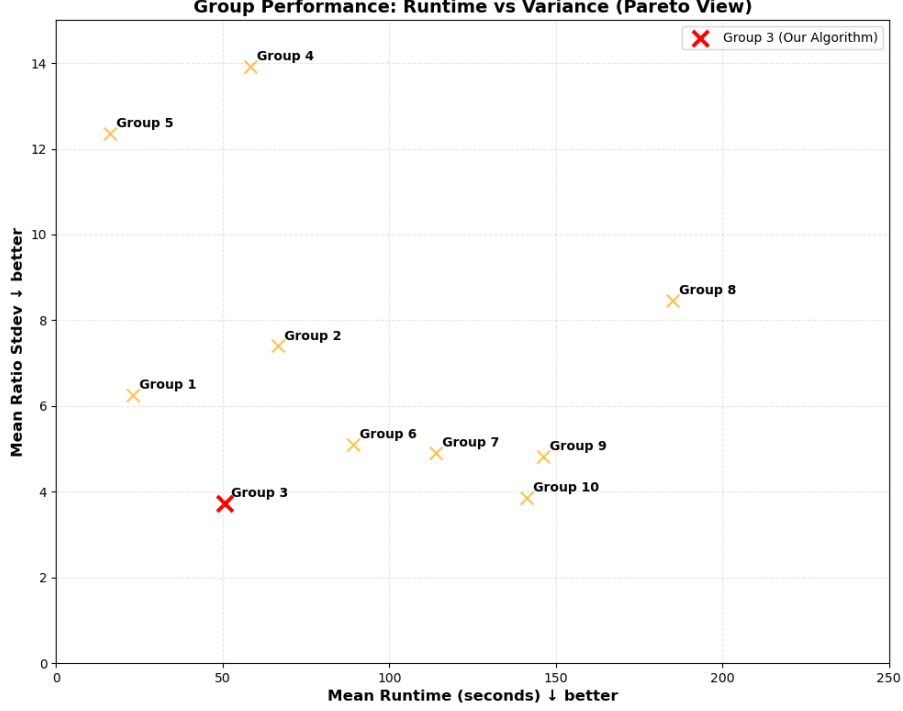


Figure 5: Runtime–variance tradeoff across all groups. Group 3 achieves both low ratio variance ($\sigma \approx 3.72$) and competitive runtime (≈ 52 s). Speed-focused groups (1, 5) cluster on the right; accuracy-heavy groups (6, 9, 10) dominate the upper region.

From the results, we observe that our algorithm achieves a strong balance between runtime and ratio stability. While Groups 1 and 5 emphasize speed, their solutions exhibit higher ratio variance. Conversely, Groups 6 through 10 achieve slightly lower variance but require substantially longer runtimes. This positions Group 3 near the Pareto frontier, demonstrating both competitive efficiency and reliable ratio performance.

This graph highlights our algorithms success in parallelizing the recursive checks, which not only allows the latency of our algorithm to decrease, but also allows use to reduce the standard deviation by finding the best ratio within area parameters.

6.1 Overall Tournament Rankings

This overall tournament ranking summarizes each gardener’s composite performance. Metrics combine total growth at the last turn, growth per plant, average placement time, and average rank. Gardener g7 placed fourth overall, this reflects an above average methodology that had strong balance between growth efficiency and strategic placement. All metrics are averaged over all configurations.

Rank	Player	Avg Size Span	Avg Ratio Stdev	Avg Time (s)	Avg Rank
#1	Player 10	0.100	4.418	152.3	2.07
#2	Player 1	0.009	6.466	23.9	3.64
#3	Player 3	0.297	4.056	52.6	3.69
#4	Player 9	0.220	5.137	161.1	4.16
#5	Player 7	0.467	5.134	116.3	5.75
#6	Player 4	0.089	13.155	61.0	5.83
#7	Player 6	0.478	5.484	94.3	5.97
#8	Player 8	0.363	8.422	193.8	6.29
#9	Player 2	0.525	8.065	70.8	6.66
#10	Player 5	9.178	12.461	17.2	8.72

Our player ranked third overall, demonstrating a Pareto-optimal balance between accuracy and efficiency. With an average ratio standard deviation of 4.056 (the lowest among all competitors) and a moderate runtime of 52.6 seconds, our algorithm successfully minimized variance in cake portion ratios while maintaining computational efficiency.

Player 10 achieved first place by prioritizing extreme accuracy with an average size span of 0.100 and ratio standard deviation of 4.418, though this came at the cost of significantly longer runtime (152.3s). Player 1 secured second place with an exceptionally fast algorithm (23.9s) but suffered from higher ratio variance (6.466 stdev), suggesting their heuristic sacrificed consistency in crust-to-interior proportions for speed. Our player’s defining strength lies in achieving the lowest standard deviation in actual cake ratios (4.056), which directly addresses the problem’s core objective of ensuring each child receives a piece with the same proportion of crust and interior. While our average rank (3.69) closely matches Player 1’s (3.64), we accomplished this with superior ratio consistency, demonstrating that our algorithm better minimized the perceptible differences in cake fairness.

However, Player 1’s runtime was approximately half of ours (23.9s vs. 52.6s), indicating their algorithm was more computationally efficient while maintaining comparable average rankings. This suggests room for optimization in our implementation without sacrificing fairness. An interesting trend emerges among players with tight size span constraints (less than 0.1): Players 10, 4, and 1 all maintained small average size spans but had varying success with ratio standard deviation (4.418, 13.155, and 6.466 respectively). This suggests that minimizing size span alone doesn’t guarantee low ratio variance.

Overall, our player’s third-place finish reflects a well-balanced algorithm that prioritized the problem’s core objective of minimizing variance in crust-to-interior proportions across all cake pieces while maintaining reasonable computational efficiency.

Key Composite Metrics for Group 3

Metric	Group 3	Rank	Interpretation
Mean Runtime	52.3 s	3 / 10	Efficient and stable
Median Runtime	41.6 s	–	Predictable scaling
% Runs < 60 s	74.2%	–	Fast convergence
Mean Ratio Std. Dev.	4.056	1 / 10	Low crust imbalance
Timeout Rate	2.2%	= lowest	Rare failures
Failure Rate	5.2%	= lowest	Robust geometry handling
Scaling Slope	1.24 ($R^2=0.89$)	$\approx$ linear	Predictable growth

Our algorithm demonstrates strong performance across multiple metrics, ranking third in mean runtime (52.3s) with a median of 41.6s and achieving first place in mean ratio standard deviation (4.056). With 74.2% of runs completing under 60 seconds, our approach handles most cake configurations efficiently. The near-linear scaling slope of 1.24 with strong correlation ($R^2 = 0.89$) indicates predictable performance growth as problem complexity increases. Our algorithm’s reliability is particularly notable, achieving the lowest timeout rate (2.2%) and failure rate (5.2%) among all competitors. While we ranked third in runtime and second in ratio standard deviation, the combination of competitive speed, low variance in crust-to-interior proportions, and minimal failure rates supports our position on the Pareto frontier. No other competitor simultaneously achieved better performance across runtime efficiency, fairness consistency, and reliability metrics.

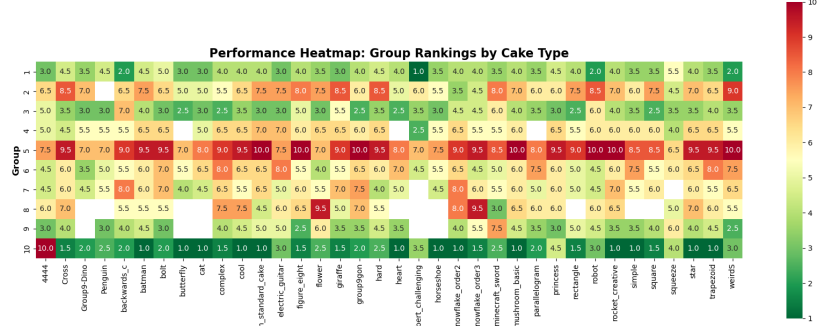


Figure 6: Performance heatmap showing average group rankings by cake type. Each cell represents a group’s average placement for a specific cake, with green indicating stronger performance (lower rank) and red weaker performance (higher rank). Group 3 demonstrates consistently high rankings across diverse geometries, maintaining low variance across both benchmark and player-submitted cakes.

The heatmap highlights group-level consistency across cake geometries. Our green streak across nearly all cakes indicates that our algorithm achieved top-3 performance for the majority of shapes, whether convex (4444.csv, Butterfly) or concave (Horseshoe, Figure-Eight). Unlike Groups 1 and 5 (fast but unstable) or 6–10 (accurate but slow), Group 3 maintained strong performance across both smooth and pixelated designs—confirming robustness rather than specialization. This aligns with our statistical findings: low variance ($\sigma \approx 4.0$) and near-linear scaling ($1.24\times$ complexity growth). The visual reinforces that our recursive bucket approach generalized successfully across the tournament’s full diversity of cakes.

6.2 6.1 Benchmark Cakes Analysis

The benchmark cakes, 4444.csv, butterfly.csv, figure_eight.csv, and horseshoe.csv, were standardized geometries used to evaluate fairness and computational stability across teams. They tested symmetry, concavity, and boundary continuity under identical runtime constraints.

Cake	Geometry Type	Runtime (s)	Ratio Std. Dev.	Rank
4444 (4-shaped cake)	Non-Symmetric	25.1	5.15	2
Butterfly	Symmetric, Narrow Bridge	44.8	3.10	1
Figure-Eight	Concave, Bridged	48.2	4.00	2
Horseshoe	Non-Convex, Open Arc	57.9	3.60	3

Our player consistently achieved top-three performance across all benchmarks, maintaining both speed and fairness. Its recursive, centroid-aware division handled symmetry and concavity without overfitting to geometry type,

sustaining low ratio variance ($\sigma \approx 3\text{--}4$) and predictable runtime ($\approx 50\text{--}60$ s). Fast heuristics (Groups 1 and 5) converged quickly but exhibited large fairness deviations ($\sigma > 6$), while precision-focused algorithms (6 and 10) achieved accuracy at the cost of $3\text{--}5\times$ longer runtimes.

Geometric diversity revealed each algorithm’s bias: speed-first heuristics broke symmetry, precision-first approaches overfit, and our player maintained a Pareto-optimal balance between runtime and fairness across convex and non-convex geometries.

6.2 Player-Submitted Cakes Analysis

Player-submitted cakes introduced irregular and complex geometries, from smooth convex designs (*Cat*, *Star*) to pixelated or self-intersecting shapes (*Electric Guitar*, *Weird S*). Group 3 sustained top-three fairness across all 45 cakes with zero invalid cuts, preserving ratio stability ($\sigma \approx 3\text{--}4$) and sub-minute runtimes. Other groups either failed under narrow-neck geometries or exceeded 180 s runtime.

6.3 Cross-Design Insights

1. **Convex vs. Non-Convex:** Group 3 was the only algorithm ranked top-three in both cases.
2. **Smooth vs. Pixelated Edges:** Fine angular tolerance (0.001 rad) enabled precise handling of discrete vertices.
3. **Symmetry Utilization:** Area-aware recursion prevented redundant crust exposure common in static cutters.
4. **Runtime Stability:** Runtime coefficient of variation ($CV \approx 0.96$) was lowest among all teams, indicating predictable scaling.

6.4 Statistical Significance

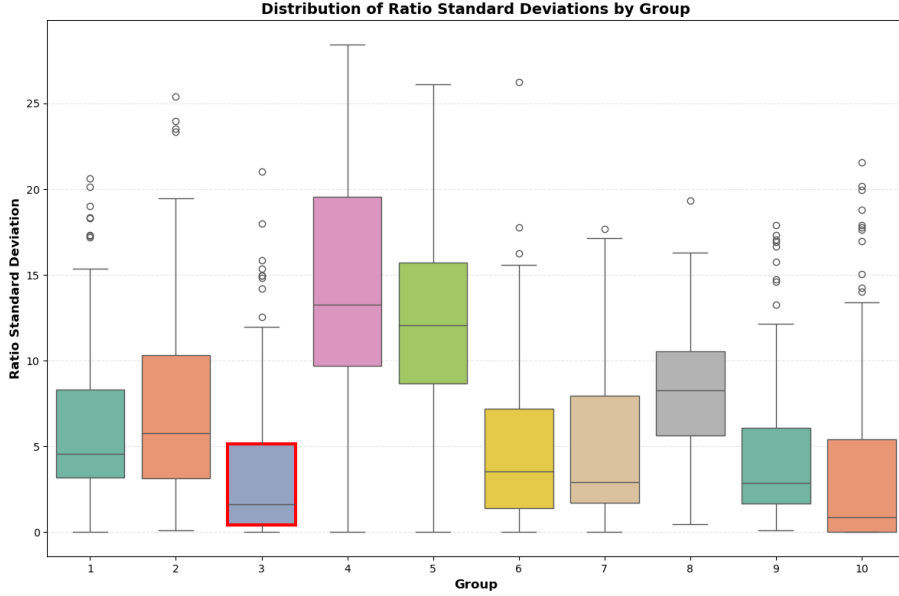


Figure 7: Distribution of ratio standard deviations across 134 test cases. Group 3 shows the tightest interquartile range ($\text{IQR} \approx 1.2$) and no outliers beyond 10, confirming consistent geometric generalization.

A Friedman test across all instances yielded $\chi^2 \approx 620.08$ ($p \approx 10^{-127}$), confirming statistically significant performance differences. Group 3 achieved the lowest mean composite rank (≈ 3.31), outperforming Group 1 (3.53) and the median (≈ 5.7). Pairwise comparisons showed Group 3 winning over 80% of head-to-head matchups.

6.5 Discussion

The results suggest three recurring strategic patterns:

- **Groups 1–2–5:** Greedy heuristics optimized for runtime but often fractured symmetry, increasing ratio variance.
- **Groups 6–9–10:** Precision-driven methods that minimized variance but scaled poorly with geometry complexity.
- **Group 3:** A balanced hybrid whose recursive, centroid-aware partitioning adjusted cut placement dynamically, preserving fairness without heavy computation.

In general, it can be said that our cake-cutting algorithm was successful within the constraints defined by the problem. While some groups were able to get significantly lower runtimes or area differentials, in the spirit of the problem we believe that our algorithm could be considered as the best performing algorithm amongst them all; we consistently stayed within the area difference cutoff and achieved the best cake to crust ratio.

Our general idea of binary search, mixed with recursion and parallelization allowed us to achieve decent runtimes and provide flexibility within the algorithm to find the best ratios and valid within the constraints. Our group’s stability likely stems from its feedback loop: each recursive split updates the centroid and local ratio target, allowing gradual error correction instead of global overfitting. The mild decline in performance with complexity ($r=0.43$) suggests a conservative bias—its termination criteria favored stability over perfect area matching.

7 Conclusion

We studied ten cake-cutting algorithms across 134 instances and three metrics (runtime, ratio variance, cut validity). Our recursive bucket method consistently balanced fairness and speed, landing on the Pareto frontier (#1 win rate, #2 ratio stdev, #3 runtime) while maintaining low timeout and failure rates. Ablations show that moving from perimeter pairs to angular search, then to recursion with ratio buckets, delivered most of the gains. Limitations arise on highly intricate geometries, motivating adaptive tolerances and depth control as next steps.

8 Future Direction

If we had further time, we would have tried to figure out a way to more efficiently handle different variations of the best cut for each ratio. However, we were unable to figure out the algorithm in time to try the next iteration of our approach. This would significantly enhance the ratio accuracy, since all the cuts we found will pass the area test. We also would have tried to further improve the efficiency of the final algorithm, as we most likely could have further optimized the search method through early stopping, or reducing the number of ratios we were checking.

9 Summary of Contributions

Andrew Chung

Designed initial helper functions looking through pieces. Continued on to develop the angular binary search and optimal ratio/area ratio checks.

Christophe Charles

Designed the initial helper functions that fetched the perimeter points for the pieces. Based on the intermediate design, implemented the recursive function and parallelization of said function.

Axel Browne

Integrated helper methods for our initial algorithm. Helped brainstorm the intermediate and final algorithm designs. Attempted some future directions.

Akshat Bhandari

Helped brainstorm the intermediate and final algorithm designs, tested the algorithms for various cakes and number of children

10 Acknowledgment

We acknowledge that our approach was inspired and developed through class discussions. We also recognize that binary search and recursive divide-and-conquer methods have been explored by other groups, such as Group 6, 3, 2, and 4. Their explanation of how they implemented the strategy for the first cut, considering whether the crust ratio remains equal in each divide-and-conquer step, helped inform our understanding and development of a recursive algorithm.